

Nepal College of Information Technology (NCIT)

Balkumari, Lalitpur | Affiliated to Pokhara University
Department of IT / Computer Engineering

LAB 3

Data Cleaning and Preliminary Visualization

Course: Data Science and Analytics (IT 405 / BUS 405)

1. Objectives

On completion of this lab, the student will be able to:

- Identify common data-quality problems in a raw dataset (missing, invalid, inconsistent and duplicate records).
- **Remove** missing records using listwise deletion when imputation is not appropriate.
- **Fill** missing values using statistical imputation (mean, median and mode).
- **Remove wrong data** — out-of-range values, invalid categories and statistical outliers.
- Produce basic visualizations to inspect and verify the dataset before and after cleaning.

2. Background Theory

Data cleaning (also called data cleansing or data wrangling) is the process of detecting and correcting corrupt, inaccurate or irrelevant records from a dataset. In a typical analytics project, data scientists spend a large share of their effort preparing data, because the reliability of every model and chart that follows depends entirely on the quality of the input. The guiding principle is “**garbage in, garbage out**” — a flawless algorithm cannot rescue conclusions drawn from dirty data.

The common categories of data-quality problems addressed in this lab are:

- **Missing data** — empty cells (NaN / null) caused by failed entry, optional fields or transmission errors.
- **Wrong / invalid data** — values that violate domain rules, e.g. a negative transaction amount or an age of 220 years.
- **Inconsistent data** — the same category written differently, e.g. “Kathmandu”, “kathmandu” and “KTM”.
- **Duplicate data** — repeated rows that bias counts and aggregates.
- **Outliers** — extreme values that distort the mean, variance and the scale of plots.

2.1 Strategies for Missing Data

Removal (deletion): rows or columns containing missing values are dropped. This is acceptable when the missing fraction is small and the records are not critical, but it discards information.

Imputation (filling): missing values are replaced with an estimate. Numeric columns are usually filled with the **mean** (symmetric data) or **median** (skewed data or data with outliers), while categorical columns are filled with the **mode** (most frequent value) or a placeholder such as “Unknown”.

3. Tools and Libraries

Library	Purpose in this lab
pandas	Loading the CSV, inspecting structure, handling missing values, filtering and de-duplication.
numpy	Numeric operations and generating the synthetic dataset.
matplotlib	Bar charts, histograms, box plots, scatter plots and the correlation heatmap.

Import statements used throughout the lab:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

4. Dataset Description

The dataset simulates digital-wallet transactions from a Nepali payment platform (eSewa-style). It is intentionally seeded with data-quality problems so that each cleaning technique can be demonstrated. It contains 123 rows (including injected duplicates) and 7 columns.

Column	Type	Description
CustomerID	Integer	Unique identifier for each customer (1001–1120).
Name	Text	Customer first name.
Age	Numeric	Customer age in years (contains missing & impossible values).
City	Category	City of the customer (contains missing & inconsistent spellings).
Product	Category	Service paid for (recharge, bills, tickets, etc.).
Amount_NPR	Numeric	Transaction amount in Nepali Rupees (contains missing, negative & extreme values).
Rating	Numeric	Service rating 1–5 (contains missing & out-of-scale values).

Loading the dataset:

```
df = pd.read_csv("esewa_transactions_raw.csv")
print(df.shape)
print(df.head())
```

Console output

```
(123, 7)
  CustomerID  Name  Age  City  Product  Amount_NPR  Rating
0        1001  Hari  45.0  Butwal  Mobile Recharge    1811.0    1.0
1        1002  Anita  24.0  Lalitpur  Mobile Recharge    2042.0    3.0
2        1003  Kiran  26.0  Bhaktapur  Mobile Recharge     851.0    4.0
3        1004  Gita  220.0  Pokhara  Internet Bill    1458.0    2.0
4        1005  Rajan  29.0  Lalitpur  Movie Ticket    -769.0    4.0
```

The very first rows already reveal problems: customer 1004 has an age of **220** and customer 1005 has a **negative amount** of **–769**. These will be handled in the wrong-data step.

5. Procedure and Observations

Step 1 — Inspecting Data Quality

Before cleaning, we profile the dataset to quantify the problems. The key diagnostic calls are `info()`, `isnull().sum()`, `duplicated().sum()` and `describe()`.

```
df.info()
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
print(df.describe())
```

Console output

Missing values per column		describe() (key statistics)			
CustomerID	0	Age	Amount_NPR	Rating	
Name	0	min	-5.0	-2300.0	1.0
Age	12	max	220.0	99999.0	9.0
City	7	mean	37.8	2393.1	3.1
Product	0				
Amount_NPR	9	Duplicate rows: 3			
Rating	15				

The profile confirms four problems: (i) **58 missing cells** across Age, City, Amount and Rating; (ii) **impossible ages** (min -5, max 220); (iii) **invalid amounts** (min -2300, max 99999); and (iv) **3 duplicate rows**. The missing values are visualized below.

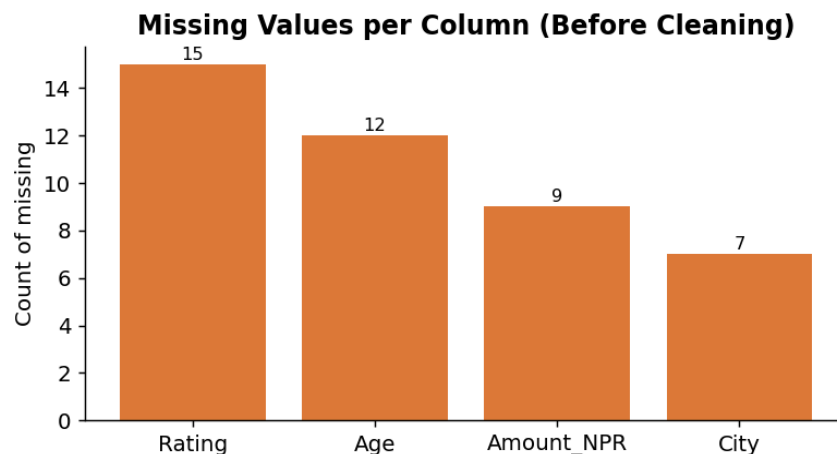


Fig. 1 — Count of missing values per column (before cleaning).

Step 2 — Removing Missing Data

When a record is missing a **critical** field that cannot be sensibly estimated, the safest option is to drop it. Here we treat **Amount_NPR** and **City** as critical: a transaction with no amount or location carries little analytical value. We use `dropna()` with the `subset` argument so that only these two columns trigger deletion.

```
# Listwise deletion: drop rows missing a critical field
df_drop = df.dropna(subset=["Amount_NPR", "City"])
print("Before:", df.shape, " After:", df_drop.shape)
```

Console output

Before: (123, 7) After: (107, 7)

Sixteen rows were removed. Deletion is simple but lossy, so we reserve it for critical columns only and prefer **imputation** (Step 3) for the remaining fields.

Step 3 — Filling Missing Data (Imputation)

For the non-critical numeric and categorical columns we retain the rows and fill the gaps. Age is filled with the median (robust to the extreme values still present), Amount with the mean, Rating with the mode, and missing City labels with the placeholder “Unknown”.

```
df_fill = df.copy()
df_fill["Age"] = df_fill["Age"].fillna(df_fill["Age"].median())
df_fill["Amount_NPR"] = df_fill["Amount_NPR"].fillna(df_fill["Amount_NPR"].mean())
df_fill["Rating"] = df_fill["Rating"].fillna(df_fill["Rating"].mode()[0])
df_fill["City"] = df_fill["City"].fillna("Unknown")

print(df_fill.isnull().sum())
```

Console output

```
CustomerID    0      Product      0
Name          0      Amount_NPR   0
Age           0      Rating       0
City          0      -> No missing values remaining
```

Note: the median is preferred for Age over the mean because the column still contains the impossible value 220, which would inflate a mean. This illustrates why wrong-data handling (Step 4) and imputation choices are linked.

Step 4 — Removing Wrong Data

“Wrong data” here covers four sub-problems handled in sequence: inconsistent categories, duplicate rows, out-of-range values and statistical outliers.

4(a) Standardizing inconsistent categories. The City column mixes cases and abbreviations (“kathmandu”, “KTM”, “ Pokhara ”). We strip whitespace, apply title-case and map abbreviations to a single canonical spelling.

```
clean = df_fill.copy()
clean["City"] = clean["City"].str.strip().str.title()
clean["City"] = clean["City"].replace({"Ktm": "Kathmandu"})
```

4(b) Removing duplicate rows.

```
before = clean.shape[0]
clean = clean.drop_duplicates()
print("Duplicates removed:", before - clean.shape[0])
```

Console output

```
Duplicates removed: 3
```

4(c) Removing out-of-range (invalid) values. Domain rules: age must be 10–90, amount must be positive, rating must be 1–5. Records violating these rules are filtered out.

```
clean = clean[(clean["Age"] >= 10) & (clean["Age"] <= 90)]
clean = clean[clean["Amount_NPR"] > 0]
clean = clean[(clean["Rating"] >= 1) & (clean["Rating"] <= 5)]
```

4(d) Removing statistical outliers (IQR method). Even after range filtering, an extreme value such as 99,999 distorts the scale. We use the **Interquartile Range (IQR)** rule: any value below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is treated as an outlier.

```
Q1, Q3 = clean["Amount_NPR"].quantile([0.25, 0.75])
IQR = Q3 - Q1
low, high = Q1 - 1.5*IQR, Q3 + 1.5*IQR
clean = clean[(clean["Amount_NPR"] >= low) & (clean["Amount_NPR"] <= high)]
print("Bounds:", round(low,1), "to", round(high,1))
print("Final shape:", clean.shape)
```

Console output

```
Bounds: -139.2 to 3428.8
Final shape: (113, 7)
```

The box plot below shows the effect: the single extreme transaction ($\approx 99,999$ NPR) dominates the “before” plot and is removed in the “after” plot, restoring a readable scale.

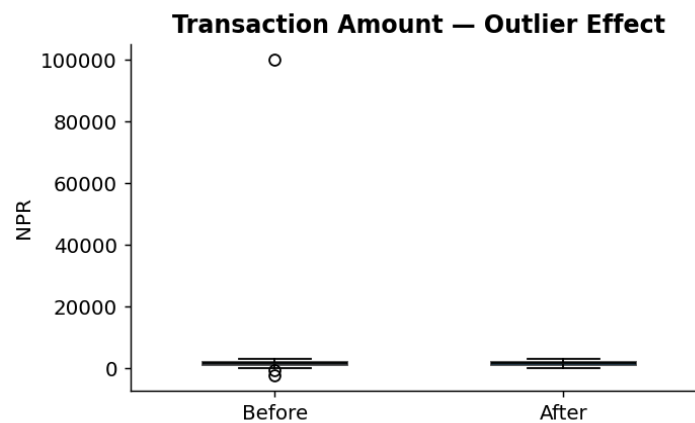


Fig. 2 — Transaction amount before and after outlier removal.

6. Basic Data Visualization

With a clean dataset we produce exploratory charts. Visualization both verifies the cleaning and surfaces patterns in the data.

6.1 Histogram — Distribution of Amount

A histogram shows how transaction amounts are distributed. After cleaning, the distribution is concentrated in a realistic range instead of being flattened by the 99,999 outlier.

```
plt.hist(clean["Amount_NPR"], bins=20, color="#2E75B6")
plt.xlabel("Amount (NPR)"); plt.ylabel("Frequency")
plt.show()
```

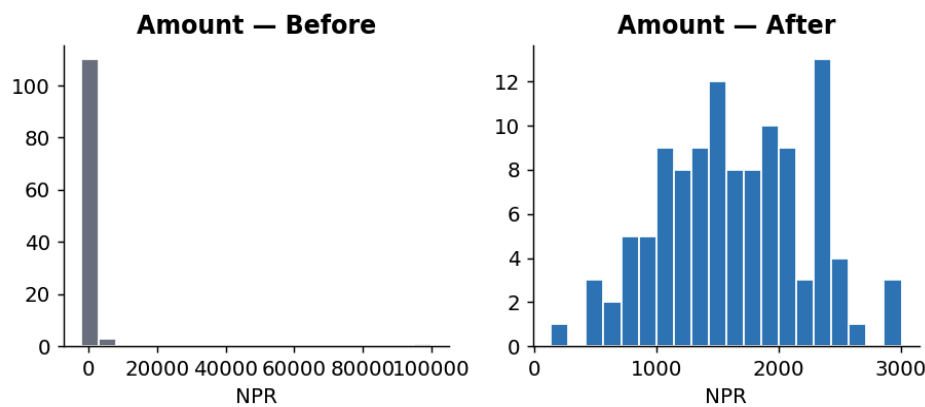


Fig. 3 — Distribution of transaction amount, before vs. after cleaning.

6.2 Bar Chart — Transactions by City

A bar chart of value counts shows how transactions are spread across cities. After standardization, Kathmandu correctly aggregates the former “kathmandu” and “KTM” entries into a single bar.

```
clean["City"].value_counts().plot(kind="bar", color="#2E75B6")
plt.ylabel("Count"); plt.show()
```

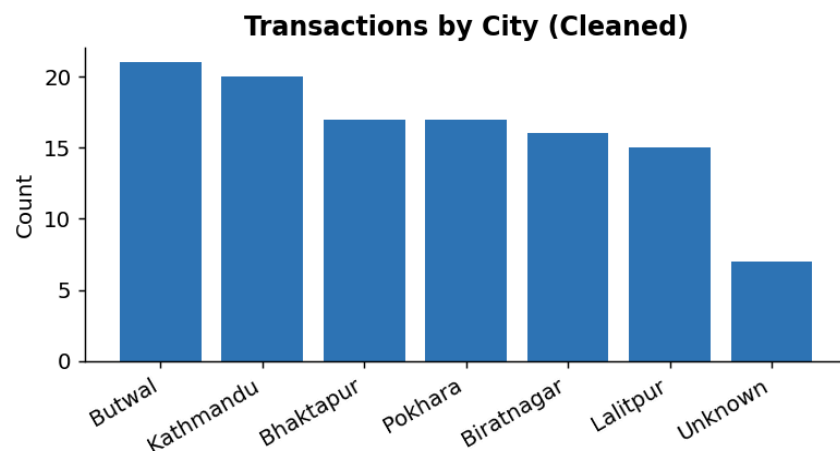


Fig. 4 — Number of transactions per city (cleaned & standardized).

6.3 Bar Chart — Average Amount by Service

Grouping by Product and averaging the amount highlights which services carry the largest transactions.

```
clean.groupby("Product")["Amount_NPR"].mean().sort_values().plot(kind="barh")
plt.xlabel("Average NPR"); plt.show()
```

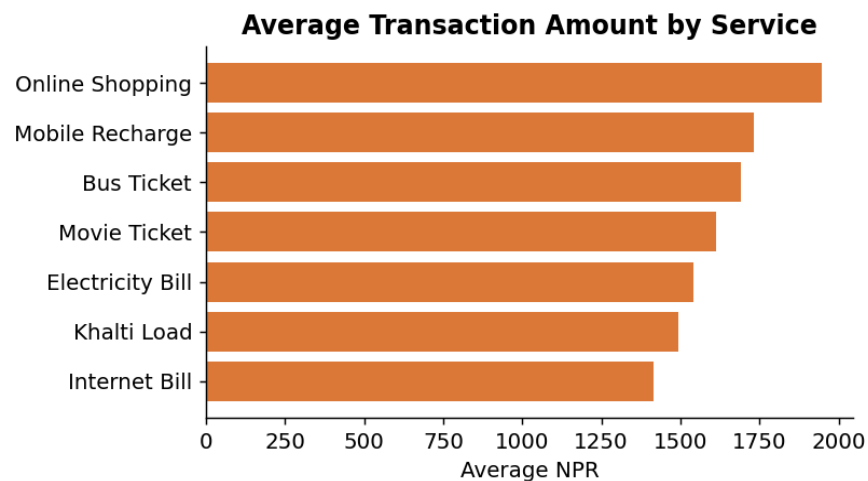


Fig. 5 — Average transaction amount by service type.

6.4 Scatter Plot — Age vs. Amount

A scatter plot examines the relationship between two numeric variables; here colour encodes the rating as a third dimension.

```
plt.scatter(clean["Age"], clean["Amount_NPR"], c=clean["Rating"], cmap="viridis")
plt.xlabel("Age"); plt.ylabel("Amount (NPR)"); plt.colorbar(label="Rating")
plt.show()
```

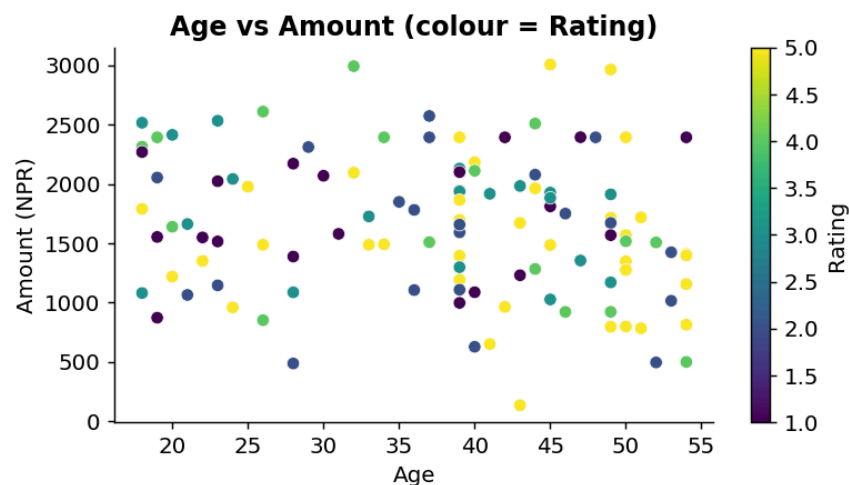


Fig. 6 — Age vs. amount, coloured by rating.

6.5 Correlation Heatmap

A heatmap of pairwise correlations summarizes linear relationships among the numeric columns. Values near 0 indicate little linear association — expected for this independently generated data.

```
corr = clean[["Age", "Amount_NPR", "Rating"]].corr()  
plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1); plt.colorbar()  
plt.show()
```

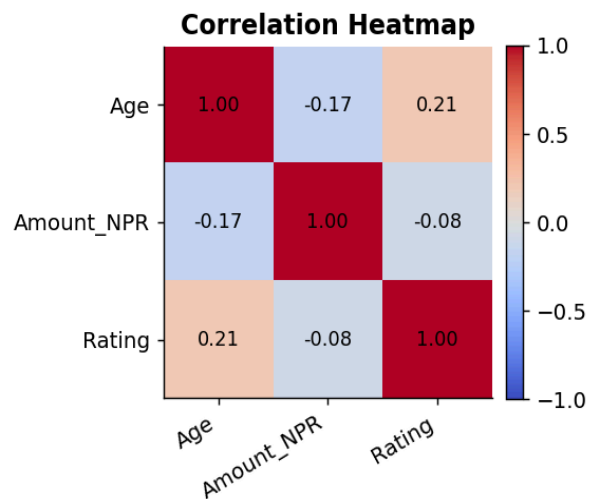


Fig. 7 — Correlation heatmap of the numeric columns.

Lab Exercise — Practice Questions

Use the **Employee dataset** given in Table 1 below to answer the following questions. The dataset contains **15 records** and deliberately includes **missing values** (shown as *NaN*), **duplicate rows** (highlighted) and **outliers**. Write Python (pandas / matplotlib) code for each task and report the output.

Table 1 — Employee dataset (NSA / NCIT staff records, synthetic)

EmpID	Name	Department	Age	Experience (Yrs)	Salary (NPR)
E01	Aarav	IT	28	4	45000
E02	Sita	HR	35	9	52000
E03	Bibek	Finance	NaN	6	48000
E04	Anita	IT	41	15	70000
E05	Rajan	Marketing	30	5	NaN
E06	Puja	HR	26	2	38000
E07	Hari	Finance	45	20	65000
E08	Gita	IT	33	8	55000
E09	Suman	Marketing	29	NaN	42000
E10	Nisha	HR	38	12	950000
E11	Kiran	Finance	31	7	50000
E12	Sabina	NaN	27	3	40000
E04	Anita	IT	41	15	70000
E13	Manish	IT	90	25	60000
E06	Puja	HR	26	2	38000

Highlighted rows are exact duplicates. NaN marks a missing value. Note the extreme Salary (950000) and Age (90).

- Q1. Creating a DataFrame.** Construct a pandas DataFrame named `df` from the data in Table 1 using a Python dictionary. Display its shape, the first five rows (`df.head()`), and the output of `df.info()`.
- Q2. Finding missing values.** Write code to (a) count the number of missing values in each column using `df.isnull().sum()`, and (b) detect and count the duplicate rows using `df.duplicated().sum()`. State which columns and rows are affected.
- Q3. Imputing missing values.** Fill the missing values appropriately: impute the numeric columns (**Age**, **Experience**, **Salary**) with the **median**, and the categorical column (**Department**) with the **mode**. Verify that no missing values remain, and explain why the median is preferred over the mean for this dataset.
- Q4. Cleaning wrong data.** Remove the duplicate rows, then handle the outliers — detect the outlier in **Salary** using the IQR rule and treat the impossible **Age** value (90). Report the shape of the dataset after cleaning.
- Q5. Standardization.** Apply **z-score standardization** to the numeric columns (Age, Experience, Salary) so each has mean 0 and standard deviation 1, using the formula $z = (x - \mu) / \sigma$ (or `StandardScaler` from `scikit-learn`). Display the first five standardized rows.
- Q6. Data transformation.** Perform the following transformations on the cleaned data:

- (a) Apply a **log transformation** to **Salary** using `np.log1p()` and store it in a new column `Salary_Log` to reduce skewness.
 - (b) **Encode** the **Department** column into numeric codes (label encoding).
 - (c) Create an `Age_Group` column by **binning** **Age** into **Young** (<30), **Mid** (30–45) and **Senior** (>45).
- Q7. Histogram.** Plot a **histogram** of the **Age** column using 8 bins. Describe the shape of the distribution.
- Q8. Box plot.** Draw a **box plot** of the **Salary** column before and after outlier removal. Identify the outlier value from the plot.
- Q9. Count plot.** Draw a **count plot** (bar chart of value counts) of the **Department** column. Which department has the most employees?
- Q10. Scatter plot.** Draw a **scatter plot** of **Experience** (x-axis) versus **Salary** (y-axis). Comment on whether there appears to be a relationship between experience and salary.